STAT6207 Assignment 2 - README

STAT6207 Applied Deep Learning — Assignment 2

Sequence Recognition: `Appliances` One-Step Forecasting

Build a time-series **one-step-ahead** forecasting model with OpenCode: using the history sampled every 10 minutes, predict the next value of household electricity consumption `Appliances` (Wh). It covers all assignment items: **Basic LSTM**, **Advanced A local LLM**, **Advanced B Transformer**, and **Advanced C generalizable pipeline**, together with the **prompts, report, session transcripts**, and **3 erroneous OpenCode code suggestions**.

Deliverables Overview (by assignment item)

Assignment item	Deliverable / Result	Location	How to find it
Basic (50%) LSTM forecast	LSTM code, metrics, prediction plots	`src/train_lstm_v2.py`, `results/lstm_v2/`	See ① Basic below
Advanced B (15%) Transformer	Transformer code, metrics, plots	`src/transformer_forecast.py`, `results/transformer/`	See ② Advanced B
Advanced C (20%) Generalizable pipeline	Generic pipeline + cross-period transfer results	`src/run_forecast.py`, `results/forecast/`	See ③ Advanced C
Advanced A (15%) Local LLM	Local-model-generated script + comparison	`local_llm/`	See ④ Advanced A
Prompt	Original prompt text for each stage	`prompts/prompts_used.md`	See ⑤ Submitted prompts
Report (PDF)	Report body	`report/report.md` (→ `report/report.pdf`)	See ⑥ Report
Session transcripts	All OpenCode session records + index	`transcripts/`	See ⑦ Session transcripts
3 code suggestions	3 erroneous suggestions + lecture basis + fixes	`report/report.md` Section 8, `report/bug_log.md`	See ⑧ Three code suggestions

> All result metrics are computed on the **original (de-normalized) Wh scale**, and are always compared against the **naive persistence** baseline.

---

① Basic (50%) — LSTM

Split by time (no shuffling), fit the scaler only on the training segment, no look-ahead, sliding window L=24, validation-set early stopping.

Model	RMSE (Wh)	vs. naive
naive persistence (baseline)	86.6	—
LSTM v1 (no validation/no early stop, counterexample)	282.4	3.3× worse
LSTM v2 (univariate + early stop)	**78.8**	**better**

- Code: `src/train_lstm_v2.py` (main result, univariate); `src/train_lstm.py` (deliberately kept overfitting counterexample).
- Data pipeline: `src/data_pipeline.py` → `artifacts/`.
- Results: `results/lstm_v2/{metrics_v2.json, lstm_pred_v2.png, lstm_loss_v2.png}`, counterexample in `results/lstm_v1_broken/`.
- **How to read**: open the prediction plot to compare truth/prediction/naive; `metrics_v2.json` holds the original-scale metrics.

② Advanced B (15%) — Transformer

Identical split, scaler, and early-stopping protocol as the LSTM, for a fair comparison under the same univariate input.

Model	#Params	RMSE (Wh)
Transformer (univariate)	100161	**78.5**
LSTM (univariate)	50497	78.8
naive persistence	—	86.6

- Code: `src/transformer_forecast.py`; results: `results/transformer/`.
- **How to read**: `metrics_transformer.json` contains the three-way Transformer/LSTM/naive comparison; `transformer_pred.png` and `transformer_loss.png` are the prediction and training curves.

③ Advanced C (20%) — Generalizable pipeline (different periods)

`src/run_forecast.py` runs on **any period / similar CSV** without code changes:

```
--csv --target --window --test-ratio --model {lstm,transformer} --start --end --tag
--features {all,univariate}
```

Transferred to a different period `data/A2_periodB.csv` (2016-01-18 → 01-29, **no overlap** with the release week):

Model	Features	RMSE	naive RMSE
LSTM	all	22.6	11.0
LSTM	univariate	**12.5**	11.0
Transformer	all	21.0	11.0
Transformer	univariate	**13.2**	11.0

- Results: `results/forecast/metrics_periodB_*.json`, `pred_periodB_*.png`.
- **How to read**: univariate nearly halves the error versus all features, yet is still slightly above naive — illustrating the **distribution shift** limitation.

④ Advanced A (15%) — Local LLM (comparison)

- Goal: use a **locally deployed LLM** to generate an LSTM script equivalent to Basic, and compare the results and code quality.
- Status: **Ollama** is installed and running on this machine (`ollama serve`, API v0.40.3).
- Artifact location: `local_llm/` (`llm_raw_output.txt`, `llm_generated.py`, `metrics.json`, plots).
- Report location: `report/report.md` **Section 5 (Advanced A)**
- **How to do/read it**: the prompt is in `prompts/prompts_used.md` under "Advanced A"; once the run finishes, compare `local_llm/metrics.json` against this project's LSTM v2 (78.8) and naive (86.6).

⑤ Submitted prompts

- `prompts/prompts_used.md`: **original prompt text for each stage** (EDA → data processing → LSTM v1/v2 → Transformer → generalizable pipeline → feature selection → local LLM), each annotated with its corresponding artifact and bug number.
- They can also be verified verbatim from `transcripts/`.

⑥ Report (PDF)

- Body: `report/report.pdf`
- Appendices: `report/EDA_report.pdf` (exploratory analysis), `report/bug_log.pdf` (error details).
- Contents: task/data, method and prompt design, LSTM, Transformer, generalizable pipeline, local LLM (placeholder), **3 code suggestions**, conclusions and limitations, reproduction commands.
- **How to read**: read in order; Section 8 directly maps to the assignment's "3 erroneous suggestions".

⑦ Session transcripts (transcripts)

- `transcripts/`: all session records (Markdown), with `transcripts/README.md` as the index table (file → stage → model → related bug number).
- **How to read**: find the stage you want in `README.md` and open the corresponding `session_*.md`; the records fully preserve the "going astray → correction" process.

⑧ Three code suggestions (required)

The 3 erroneous/suboptimal OpenCode code suggestions listed in the report (original approach → problem → lecture basis → fix → metric change):

#	Problem	After fix
#5	No validation set / no early stopping, run fixed epochs and take the last-epoch weights → overfitting	RMSE 282.4 → **78.8**
#7	Training loop used the **test set** for per-epoch monitoring (test-set leakage)	Test set fully isolated; evaluation trustworthy
#6	Generalizable pipeline fed all features without selection (including `rv1/rv2`)	RMSE 157.5 → **78.8** (univariate)

See `report/report.md` Section 8 and `report/bug_log.md` (plus 4 additional EDA/process items).

Directory structure

```
assignment_2/
├── README.md                    # this file (index of assignment results)
├── requirements.txt
├── report/
│   ├── report.md                # report body (→ report.pdf)
│   ├── EDA_report.md            # appendix: exploratory analysis
│   └── bug_log.md               # appendix: erroneous-suggestion details
├── prompts/
│   └── prompts_used.md          # original submitted prompts
├── transcripts/                 # session records + README index
├── src/                         # all code
│   ├── data_pipeline.py
│   ├── train_lstm.py            # LSTM v1 (counterexample)
│   ├── train_lstm_v2.py         # LSTM v2 (main result)
│   ├── transformer_forecast.py
│   └── run_forecast.py
├── data/                        # data
│   ├── A2_data_release.csv      # release-week data (Basic/B/C)
│   └── A2_periodB.csv           # different period (Advanced C, another period from the UCI
dataset)
├── artifacts/                   # intermediate artifacts (regenerable via data_pipeline.py)
│   ├── data_windows.npz
│   └── scaler.pkl
└── results/                     # results (plots + metrics)
    ├── eda/eda_appliances.png
    ├── lstm_v1_broken/          # LSTM v1 (counterexample)
    ├── lstm_v2/                 # LSTM v2 (main result)
    ├── transformer/
    └── forecast/                # generalizable pipeline (incl. feature-selection ablation)
```

How to run / reproduce

```
pip install -r requirements.txt

python src/data_pipeline.py
python src/train_lstm_v2.py          # LSTM main result
python src/train_lstm.py             # LSTM counterexample (v1)
python src/transformer_forecast.py   # Transformer (run train_lstm_v2.py first)
python src/run_forecast.py --csv data/A2_data_release.csv --model lstm --features univariate --tag release_lstm_uni
python src/run_forecast.py --csv data/A2_periodB.csv      --model transformer --features univariate --tag periodB_trans_uni
```

All scripts resolve paths relative to the repository root and can be run from any location. Fixed random seed 42.

Time-series correctness rules (followed throughout)

1. Split by time, never shuffle; 2. Fit the scaler only on the training segment; 3. No look-ahead (a window ending at t predicts only t+1);
2. Windows never cross split boundaries; 5. Metrics reported on the original scale; 6. Always compare against naive persistence; 7. Fixed seed 42.